

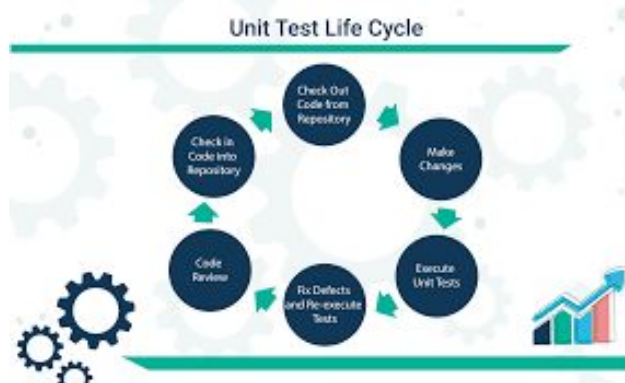
COMS W4156 Advanced Software Engineering (ASE)

October 5, 2023

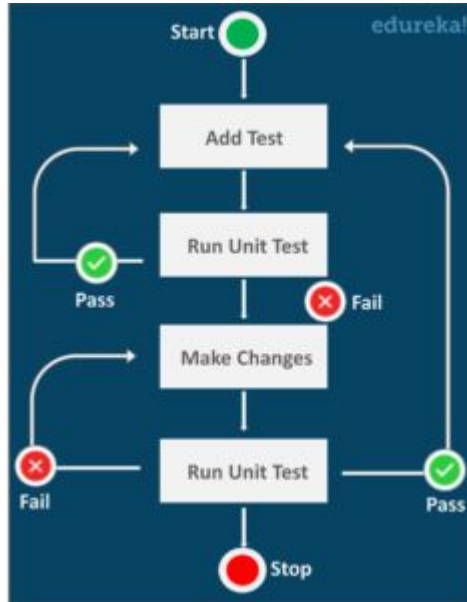


Agenda

1. unit testing
2. unit testing demo: Mengda Xu



Unit Testing



What is a “[unit](#)”? The smallest piece of code that can be logically isolated for execution

Depends on the programming language, but generally a method, function, procedure, subroutine (not a statement or expression)

Small interconnected sets of such units, such as classes, modules, files, are typically tested during unit testing as well as individual units

Interfaces between these sets are tested later, known as “[integration testing](#)” (coming soon)

Example Unit Tests

```
public class TestBadCode {  
    public int calculateFoo(int x, int y, boolean increment){  
        if(increment)  
            x++;  
            x *=2;  
        x += y;  
        return x;  
    }  
  
    @Test  
    public void test() {  
        assertEquals(calculateFoo(3, 5, true), 13);  
        assertEquals(calculateFoo(3, 5, false), 8);  
    }  
}
```

calculateFoo is the “unit under test”

This is one test class consisting of two test cases, with inputs { 3, 5, true } and { 3, 5, false }

Which of these tests pass?

Recall this is Java code, not Python

Choosing Inputs for Unit Tests

Baseline - one normal valid input (or set of inputs), one abnormal valid input, one or two invalid inputs for every non-trivial unit

Better - Baseline plus one input in middle of every equivalence partition plus 2-3 inputs at every boundary between equivalence partitions

Best? - Better plus “cover” every branch plus every loop 0, 1, 2, and many/max (if known) times around the loop



Coverage

At minimum, unit testing, integration testing and system testing collectively exercise every statement

Better: Exercise every branch (or $\geq$ 80% or 85% of branches)

Coverage may check missing conditional cases but can never detect entirely missing code

Beware the missing else!

```
if (condition) { do something }  
else { do something else }  
// some other code is here  
// some other code should be here but isn't
```

```
if (condition) { do something }  
// some other code is here  
// some other code should be here but isn't
```



Running the Tests

Now that we have chosen some inputs for one or more units under test, how do we run the tests?



Let's say we have a method that takes all its inputs as parameters and provides one output as its return value: write code that calls the method with the input parameters, receives the output, and then checks the output

- The “manual” version is create a tiny `main()` program just to call the method under test.
- Don't do it! 🤡 Instead use a unit testing framework

If the method reads/writes local shared state, such as class variables, an object and its instance variables, or an in-memory data structure like a stack or a tree: need to arrange for inputs from those sources

- The “manual” version is again create a tiny `main()` program, which now calls constructors and initializes fields, and possibly calls other methods in the same class, before calling the method under test.
- Don't do it! 🤡 Instead use a unit testing framework

What is “Local” Shared State?

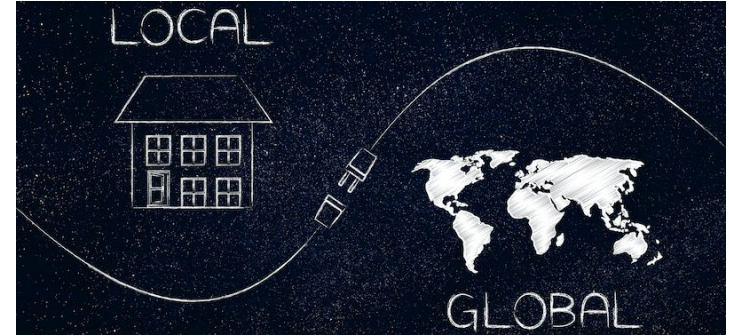
I made up this term to mean state shared only among units that are part of the same module (e.g., methods in the same class)

In an object-oriented language, this includes instance variables (object fields) and class variables (static fields)

“Global” shared state is application state shared with other code outside the module and environment state shared even outside the program, such as files, databases, devices, etc.

There may also be “global” inputs/outputs from/to UI, network, or accessing the underlying platform (e.g., time of day, random number generator)

“Global” shared state and inputs/outputs are usually faked during unit testing via [mocking](#)



Unit Testing Frameworks

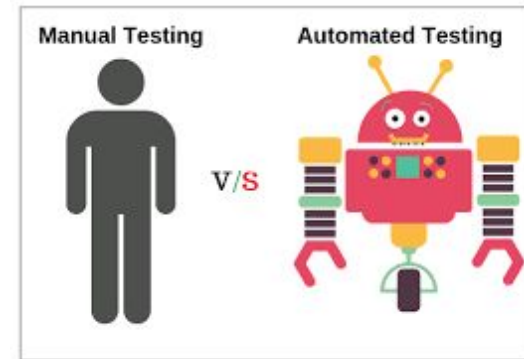
Instead of “manual” unit testing, use some unit testing framework for your programming language, e.g., [googletest](#) for C++, [junit](#) for Java, [jest](#) for Javascript

Unit testing frameworks always provide “test runners” and usually provide facilities for writing unit tests like assertions and fixtures

Unit testing frameworks often play nicely with [mocking frameworks](#), for faking “global” inputs, outputs, and shared state

Automated testing reduces human tedium and error, and is reproducible and faster so more likely to be done frequently

➤ Automated testing refers to *running* the tests, not *writing* tests (although templates may be provided), developers still write tests



Test Runner



Executes a test suite (set of test cases) in some order and reports results

Ordering might be built-in or specified by the tester - why does test order matter?

Usually does not stop when a test fails - instead runs all the tests and reports all the results

Not restricted to unit testing, works for any tests where code calls other local code

Chart	See children	Build Number ⇒ Package-Class-Testmethod names #	16	15	14	13	12	11	10	9	8	7	6	5
☐	●	org.common.samplea	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	N/A
☐	●	SampleATest	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	N/A
☐		testA	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A
☐		testB	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A
☐		testC	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	N/A
☐		testD	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A
☐	●	org.common.sampleb	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	N/A
☐	●	org.common.samplec	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A
☐	●	SampleDTest	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A
☐		testA	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A
☐		testB	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	N/A
☐		testC	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	N/A
☐		testD	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A

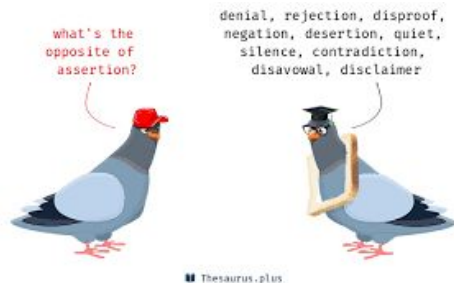
Facilities for Writing Unit Tests



Assertions = notation for checking the results (outputs and side-effects) of executing a test. Usually supports checking for true/false, checking whether an exception has been raised, and comparing a result to a known value

Test fixture = *setup* code to execute before running one or more tests, *teardown* code to run after those tests. Setup can create objects, open files or network/database connections (often involves mocks), etc. Teardown destroys objects, closes files and connections, etc.

Assertions



- **Arrange**: set up the context and prepare a bunch of objects to run the unit under test
- **Act**: call the unit under test
- **Assert**: check that outputs and side effects of the unit under test are as expected

arrange/act/assert or given/when/then

Assertions express requirements that the unit under test is expected to meet

Assertions

Notation for checking the results of executing a test (test oracle)

Usually supports checking for true/false result, comparing an output to a known value, and checking whether an exception has been raised

Method	Checks that
<code>assertEqual(a, b)</code>	<code>a == b</code>
<code>assertNotEqual(a, b)</code>	<code>a != b</code>
<code>assertTrue(x)</code>	<code>bool(x) is True</code>
<code>assertFalse(x)</code>	<code>bool(x) is False</code>
<code>assertIs(a, b)</code>	<code>a is b</code>
<code>assertIsNot(a, b)</code>	<code>a is not b</code>
<code>assertIsNone(x)</code>	<code>x is None</code>
<code>assertIsNotNone(x)</code>	<code>x is not None</code>
<code>assertIn(a, b)</code>	<code>a in b</code>
<code>assertNotIn(a, b)</code>	<code>a not in b</code>
<code>assertIsInstance(a, b)</code>	<code>isinstance(a, b)</code>
<code>assertNotIsInstance(a, b)</code>	<code>not isinstance(a, b)</code>

Example

Consider this variant of concat:

```
public class StringUtil {  
    public String concat(String a,String b) {  
        return a + b;  
    }  
}
```

Here is one approach to testing concat:

```
@Test  
public void testStringUtil_Bad() {  
    String result = StringUtil.concat("Hello ", "World");  
    System.out.println("Result is "+result);  
}
```

Why would it be difficult to automate this test?



Example

An automated testing system would always treat the first test as “pass”, since there are no assertions, and a human has to check the result

Consider concat again:

```
public class StringUtil {  
    public String concat(String a,String b) {  
        return a + b;  
    }  
}
```



Here is another approach to testing concat:

@Test

```
public void testStringUtil_Good() {  
    String result = StringUtil.concat("Hello ", "World");  
    assertEquals("Hello World", result);  
}
```

This test will “fail” if the method returns the wrong result, without human intervention



What Is Wrong With These Assertions?

```
@Test
public void getEmployeesGreaterThan25() {

    // arrange
    Employee mark = new Employee ("Mark", 19);
    Employee nina = new Employee ("Nina", 26);
    Employee jules = new Employee ("Jules", 31);
    clearDatabase();
    insertEmployee(mark);
    insertEmployee(nina);
    insertEmployee(jules);
    Predicate<Employee> constraint = e -> e.getAge() > 25;

    // act
    List<Employee> result = getEmployees(constraint);

    // assert
    assertEquals(result.size(), 2);
    assertEquals(result.get(0).getName(), "Nina");
    assertEquals(result.get(0).getAge(), 26);
    assertEquals(result.get(1).getName(), "Jules");
    assertEquals(result.get(1).getAge(), 31);
}
```

```
for (Employee e : result) {
    assertTrue(e.getAge() > 25);
}
```

```
boolean first = result.get(0).getAge() > 25;
boolean second = result.get(1).getAge() > 25;
assertTrue(first && second);
```

```
assertTrue(result.get(0).getAge() > 25);
assertTrue(result.get(1).getAge() > 25);
```


Good Assertions



Tests are software too, they may have bugs and they need to be robust, maintainable and effective

Good assertions check what is precisely requested from the code under test, as opposed to checking an overly precise or an overly loose condition

Don't write assertions which check conditions that could be split into multiple assert statements, always check the simplest condition possible

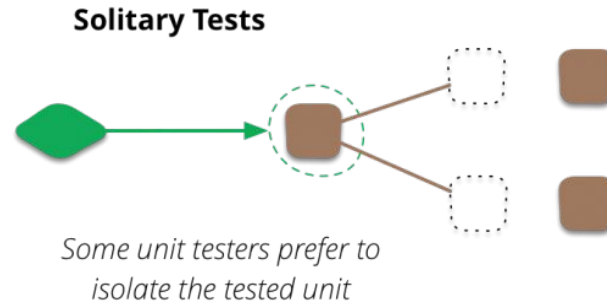
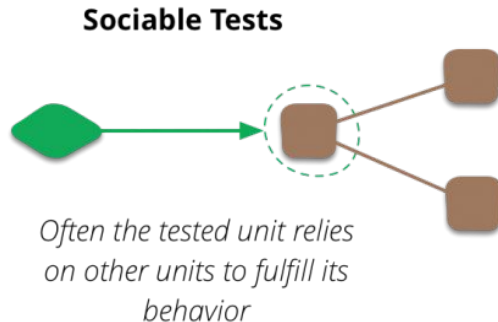
Write assertions that check requirements, not implementation details of the unit under test. Developers do not want to update a test just because the unit under test has changed for reasons unrelated to the test

Unit Testing in Isolation

Unit tests usually aim to test a method or class *in isolation from the rest of the program*

The method, or a series of methods in the same class, is called from a test runner or another kind of test driver, not from the other code in the rest of the program

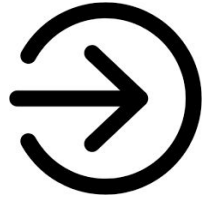
And when the methods under test would normally call other code in the rest of the program outside the class, that other code is usually faked (mocked)



Test **Every** Entry Point

Units (methods, functions) typically have parameters or arguments provided in call

But parameters are not the only [entry points](#)!



- Global variables
- Return values and side-effects from API calls are also inputs
- Data entered via GUI or CLI are also inputs
- Data retrieved from network, databases, files, devices, etc. are also inputs

When testing units *in isolation*, need some way to specify the inputs for these other kinds of entry points - test doubles (mocks)

Fixtures

A test fixture is a fixed state of a set of objects used as a baseline for running tests. The purpose of a test fixture is to ensure that there is a well known and fixed environment in which tests are run so that results are repeatable. Typically applied to a test class with multiple test cases, not just a single test case

- Preparation of input data and setup/creation of fake or mock objects
- Loading a database with a specific, known set of data
- Copying a specific known set of files

Good test fixtures also clean up after themselves!



Test Fixtures

Setup before running group of tests, teardown after running those tests (leave clean state for the next group of tests!)

@BeforeEach, @BeforeAll,
@AfterEach, @AfterAll

```
class StandardTests {  
    @BeforeAll  
    static void initAll() {  
    }  
    @BeforeEach  
    void init() {  
    }  
}
```

```
@Test  
void succeedingTest() {  
}  
@Test  
void failingTest() {  
    fail("a failing test");  
}  
@Test  
@Disabled("for demonstration purposes")  
void skippedTest() {  
    // not executed  
}  
@AfterEach  
void tearDown() {  
}  
@AfterAll  
static void tearDownAll() {  
}
```

What Does This Do?

```
import java.io.Closeable;
import java.io.IOException;

import org.junit.After;
import org.junit.AfterClass;
import org.junit.Before;
import org.junit.BeforeClass;
import org.junit.Test;

public class TestFixturesExample {
    static class ExpensiveManagedResource implements Closeable {
        @Override
        public void close() throws IOException {}
    }

    static class ManagedResource implements Closeable {
        @Override
        public void close() throws IOException {}
    }

    @BeforeClass
    public static void setUpClass() {
        System.out.println("@BeforeClass setUpClass");
        myExpensiveManagedResource = new ExpensiveManagedResource();
    }

    @AfterClass
    public static void tearDownClass() throws IOException {
        System.out.println("@AfterClass tearDownClass");
        myExpensiveManagedResource.close();
        myExpensiveManagedResource = null;
    }
}
```

```
private ManagedResource myManagedResource;
private static ExpensiveManagedResource myExpensiveManagedResource;

private void println(String string) {
    System.out.println(string);
}

@Before
public void setUp() {
    this.println("@Before setUp");
    this.myManagedResource = new ManagedResource();
}

@After
public void tearDown() throws IOException {
    this.println("@After tearDown");
    this.myManagedResource.close();
    this.myManagedResource = null;
}

@Test
public void test1() {
    this.println("@Test test1()");
}

@Test
public void test2() {
    this.println("@Test test2()");
}
}
```

Fixtures setUp and TearDown

```
import java.io.Closeable;
import java.io.IOException;

import org.junit.After;
import org.junit.AfterClass;
import org.junit.Before;
import org.junit.BeforeClass;
import org.junit.Test;

public class TestFixturesExample {
    static class ExpensiveManagedResource implements Closeable {
        @Override
        public void close() throws IOException {}
    }

    static class ManagedResource implements Closeable {
        @Override
        public void close() throws IOException {}
    }

    @BeforeClass
    public static void setUpClass() {
        System.out.println("@BeforeClass setUpClass");
        myExpensiveManagedResource = new ExpensiveManagedResource();
    }

    @AfterClass
    public static void tearDownClass() throws IOException {
        System.out.println("@AfterClass tearDownClass");
        myExpensiveManagedResource.close();
        myExpensiveManagedResource = null;
    }
}
```

```
private ManagedResource myManagedResource;
private static ExpensiveManagedResource myExpensiveManagedResource;

private void println(String string) {
    System.out.println(string);
}

@Before
public void setUp() {
    this.println("@Before setUp");
    this.myManagedResource = new ManagedResource();
}

@After
public void tearDown() throws IOException {
    this.println("@After tearDown");
    this.myManagedResource.close();
    this.myManagedResource = null;
}

@Test
public void test1() {
    this.println("@Test test1()");
}

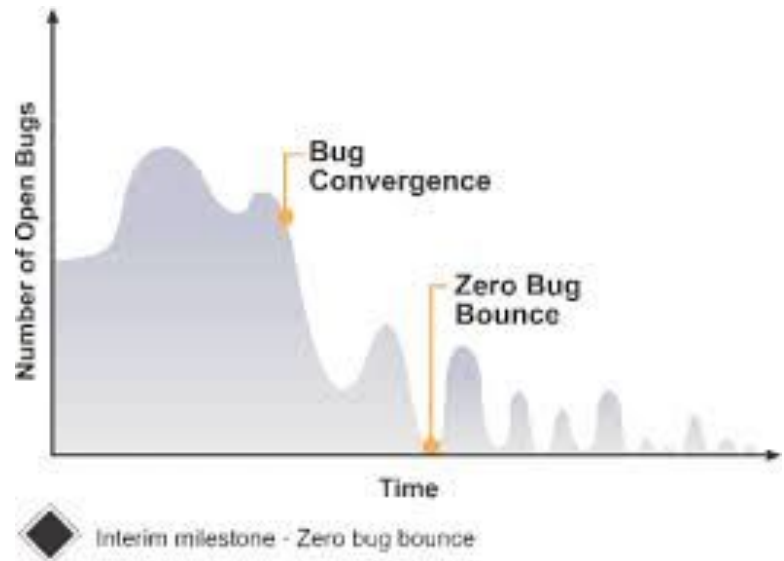
@Test
public void test2() {
    this.println("@Test test2()");
}
}
```

```
@BeforeClass setUpClass
@Before setUp
@Test test2()
@After tearDown
@Before setUp
@Test test1()
@After tearDown
@AfterClass tearDownClass
```

How Do We Know When We Are Done Testing?

- We're *never* done if we keep changing the code (regression testing)
- This is why many software organizations “freeze” a planned release build a week or more in advance, with no changes allowed except bug fixes (with each set of repairs followed by full re-testing)

Zero bug bounce: Rate of finding new bugs is stable and low (approaching zero), reached point of diminishing returns



Agenda

1. unit testing
2. unit testing demo: Mengda Xu



unit testing demo: Mengda Xu

<https://github.com/mengdaxu/unit-test-demo>

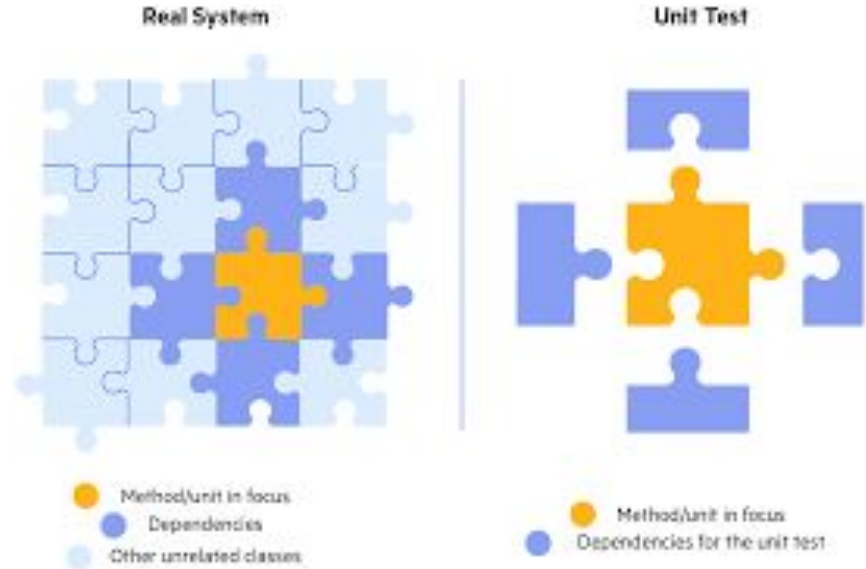
Upcoming Assignments: Note extended deadlines!

- [Team Revised Project Proposal](#) due tonight by 11:59pm
- [First Iteration](#) due Thursday, October 19, by 11:59pm
- [First Iteration Demo](#) due Thursday, October 26, by 11:59pm



Next Week

mocks and integration testing



Ask Me Anything